

Sistemas de Tiempo Real – Gestión de Tareas y Concurrency

Tomás Romanutti – 2/5/2026

Desafío 1: Concurrency Básica

Código relevante

```
void vTask1Ch1(void *pvParameters){
    while(1){
        HAL_GPIO_TogglePin(GPIOD, LD4_Pin);
        HAL_Delay(50);
    }
};

void vTask2Ch1(void *pvParameters){
    while(1){
        HAL_GPIO_TogglePin(GPIOD, LD3_Pin);
        HAL_Delay(100);
    }
};

void vTask3Ch1(void *pvParameters){
    while(1){
        HAL_GPIO_TogglePin(GPIOD, LD5_Pin);
        HAL_Delay(200);
    }
};

void vTask4Ch1(void *pvParameters){
    while(1){
        HAL_GPIO_TogglePin(GPIOD, LD6_Pin);
        HAL_Delay(400);
    }
};

void vChallenge1(){
    xTaskCreate(vTask1Ch1, "Task1", 100, NULL, 1, NULL);
    xTaskCreate(vTask2Ch1, "Task2", 100, NULL, 1, NULL);
    xTaskCreate(vTask3Ch1, "Task3", 100, NULL, 1, NULL);
    xTaskCreate(vTask4Ch1, "Task4", 100, NULL, 1, NULL);
};
```

Análisis

Todas las tareas se ejecutan? Hay concurrency entre tareas? Porque? Los tiempos de conmutación de los leds son precisos?

Incluso con todas las tareas realizando esperas no bloqueantes, todas se ejecutan con normalidad. Esto se debe a que el sistema operativo está configurado con un *preemptive scheduler* que le quita el

control a las tareas cada cierto período de tiempo. En particular cada *tick* del sistema operativo (1ms), el *scheduler* cambia el contexto entre la tarea que se ejecutó ese *tick* con alguna de las otras, de tal manera que todas mantengan equitativamente control del hilo de ejecución. Por este dato se puede concluir que los tiempos de conmutación de los leds serán precisos en períodos de hasta 3ms de duración (si una tarea tiene un período más corto, deberá esperar a que las otras tres se ejecuten por 1ms cada una para conmutar el led).

Desafío 2: Instanciación Múltiple y Paso de Parámetros

Código relevante

```
const typedef struct {
    GPIO_TypeDef* port;
    uint16_t pin;
    uint16_t delay;
} toggleInstruction_typedef;

void vTareaParpadeo(void *pvParameters){
    toggleInstruction_typedef *specific_pin = pvParameters;
    TickType_t delay = pdMS_TO_TICKS(specific_pin->delay);
    while(1){
        HAL_GPIO_TogglePin(specific_pin->port, specific_pin->pin);
        vTaskDelay(delay);
    };
};

static const toggleInstruction_typedef instructionsCh2[4] = {{GPIOD, LD4_Pin, 50},
{GPIOD, LD3_Pin, 100}, {GPIOD, LD5_Pin, 200}, {GPIOD, LD6_Pin, 400}};

void vChallenge2(){
    xTaskCreate(vTareaParpadeo, "Task1", 100, (void *) &instructionsCh2[0], 1, NULL);
    xTaskCreate(vTareaParpadeo, "Task2", 100, (void *) &instructionsCh2[1], 1, NULL);
    xTaskCreate(vTareaParpadeo, "Task3", 100, (void *) &instructionsCh2[2], 1, NULL);
    xTaskCreate(vTareaParpadeo, "Task4", 100, (void *) &instructionsCh2[3], 1, NULL);
};
```

Análisis

Que diferencias hay en las transiciones de tareas respecto al caso del Desafío 1?

El cambio a funciones de espera bloqueantes reduce el período que permite a las tareas permanecer precisas. Como cada tarea cede el control una vez termina de conmutar el led (acción que requiere muy poco tiempo), esto permite períodos menores a 3ms debido a que las tareas ceden el control antes que el *scheduler* tenga que intervenir, por lo que libera ese tiempo de espera hasta que pase un *tick* del sistema operativo.

Desafío 3: Polling vs. Bloqueo Eficiente

Código relevante

```
void vTaskLEDCh3(void *pvParameters){
    toggleInstruction_ttypedef *instruction = pvParameters;
    while(1){
        HAL_GPIO_TogglePin(instruction->port, instruction->pin);
        HAL_Delay(instruction->delay);
    }
}

void vTaskButtonCh3(void *pvParameters){
    toggleInstruction_ttypedef *instruction = pvParameters;
    TickType_t delay = pdMS_TO_TICKS(instruction->delay); // This delay isn't for
    // blinking the led, but for waking up the task for the polling.
    while(1){
        GPIO_PinState buttonState = HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0);

        if(buttonState == GPIO_PIN_SET){
            HAL_GPIO_WritePin(instruction->port,instruction->pin,
GPIO_PIN_SET);
        }
        if(buttonState == GPIO_PIN_RESET){
            HAL_GPIO_WritePin(instruction->port,instruction->pin,
GPIO_PIN_RESET);
        }
        vTaskDelay(delay);
    }
}

const toggleInstruction_ttypedef instructionsCh3[2] = {{GPIOD, LD4_Pin, 500}, {GPIOD,
LD3_Pin, 10}};
void vChallenge3(){
    xTaskCreate(vTaskLEDCh3, "LED task", 100, (void *) &instructionsCh3[0], 0, NULL);
    xTaskCreate(vTaskButtonCh3, "Button task", 100, (void *) &instructionsCh3[1], 1,
    NULL);
}
```

La definición de `toggleInstruction_ttypedef` se encuentra en el desafío 2, y fue ignorada para eliminar redundancia

Análisis

Probar con diferentes tiempos para `vTaskDelay()`. Que efecto se observa a medida que ese tiempo aumenta? Que tiempo satisface el deadline impuesto en la consigna?

El tiempo que se pasa como parámetro en `vTaskDelay` de la tarea de *polling* se corresponde con el tiempo de respuesta del led. Mientras mayor es este tiempo, mayor es el tiempo máximo de respuesta del led (puede suceder que se apriete el botón justo antes que la tarea vuelva de la espera, dando la ilusión de que tiene un tiempo de respuesta bajo). Como el tiempo de respuesta propuesto no puede superar los 10ms, fijar el período de *polling* en 10ms es suficiente (técnicamente, esto causa que el tiempo máximo de respuesta sea 10ms + el tiempo que demora el led en prenderse, pero este tiempo es despreciable).

Desafío 4: Periodicidad Absoluta vs. Relativa (El problema de la Deriva)

Código relevante

```
void vTask1Ch4(void *pvParameters){
    toggleInstruction_ttypedef *specific_pin = pvParameters;
    TickType_t delay = pdMS_TO_TICKS(specific_pin->delay);
    while(1){
        HAL_GPIO_TogglePin(specific_pin->port, specific_pin->pin);
        HAL_DELAY(100);
        vTaskDelay(delay);
    };
}

void vTask2Ch4(void *pvParameters){
    toggleInstruction_ttypedef *specific_pin = pvParameters;
    TickType_t tickCount = xTaskGetTickCount();
    TickType_t delay = pdMS_TO_TICKS(specific_pin->delay);
    while(1){
        HAL_GPIO_TogglePin(specific_pin->port, specific_pin->pin);
        HAL_DELAY(100);
        vTaskDelayUntil(&tickCount,delay);
    };
}

const toggleInstruction_ttypedef instructionsCh4[2] = {{GPIOD, LD4_Pin, 400}, {GPIOD, LD3_Pin, 500}};

void vChallenge4(){
    xTaskCreate(vTask1Ch4, "task1", 100, (void *) &instructionsCh4[0], 0, NULL);
    xTaskCreate(vTask2Ch4, "task2", 100, (void *) &instructionsCh4[1], 0, NULL);
}
```

Análisis

Intuitivamente determinar el tiempo (?) argumento de los servicios xTaskDelay para cumplir con los tiempos impuestos en la consigna. Ambos argumentos son iguales? Que efecto se observa si lo fueran? Porque?

Al tiempo de descanso de la primera tarea (que hace **vTaskDelay**) se le debe restar el tiempo de **HAL_DELAY** para estar coordinados con la segunda tarea (que hace **vTaskDelayUntil**). No son iguales debido a la diferencia entre periodicidad absoluta y relativa: la primera tarea hace que la espera sea la misma, mientras que la segunda tarea hace que el período sea el mismo. Hacer estos tiempos iguales causa que las tareas pierdan el sincronismo, haciendo que la primera tarea desplace su período 100ms cada vez que se ejecute.

Desafío 5: Control de Prioridades Dinámico

Código relevante

```
const typedef struct{
    GPIO_TypeDef* port;
    uint16_t pin;
    uint16_t delay;
    TaskHandle_t *task;
} toggleReadTaskInstruction_typedef;

void vTask1Ch5(void *pvParameters){

    toggleReadTaskInstruction_typedef *instruction = pvParameters;
    uint16_t actual_priority = uxTaskPriorityGet(NULL);
    uint16_t button_interval = pdMS_TO_TICKS(3000);

    uint16_t time = 0;
    uint16_t button_time = 0;
    uint16_t target_priority = 0;
    uint16_t waiting = pdFALSE;
    uint16_t interval = 0;
    GPIO_PinState buttonState = GPIO_PIN_RESET;

    while(1){
        buttonState = HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0);

        if(buttonState != GPIO_PIN_RESET){
            target_priority = uxTaskPriorityGet(*(instruction->task));
            vTaskPrioritySet(NULL, target_priority + (uint16_t) 1);
            button_time = HAL_GetTick();
            waiting = pdTRUE;
        }

        if(waiting == pdTRUE){
            time = HAL_GetTick();
            interval = time - button_time;
            if(interval >= button_interval){
                vTaskPrioritySet(NULL, actual_priority);
                waiting = pdFALSE;
            }
        }
        HAL_GPIO_TogglePin(instruction->port, instruction->pin);

        HAL_Delay(instruction->delay); // Not vTaskDelay!
    }
}

void vChallenge5(){
    static TaskHandle_t task2Handler = NULL;
    static const toggleInstruction_typedef task2_instruction = {GPIOD, LD5_Pin, 50};
    static toggleReadTaskInstruction_typedef task1_instruction = {GPIOD, LD4_Pin,
300, &task2Handler};
    xTaskCreate(vTareaParpadeo,"Task2",100,(void *)
&task2_instruction,1,&task2Handler);
    xTaskCreate(vTask1Ch5,"Task1",100,(void *) &task1_instruction,1,NULL);
}
```

Análisis

Que ocurre cuando la prioridad de la tarea A se reestablece? Que servicio se utilizo para hacer el conteo de los 3 segundos? Se logró el comportamiento descrito en el enunciado en el primer intento? De no ser así, porque?

Lo particular de este ejercicio es el uso de funciones de espera no bloqueantes (**HAL_DELAY**) en vez de bloqueantes (**vTaskDelay**). Esto ocurre debido a los requisitos del desafío: en particular, que la segunda tarea tenga que permanecer congelada durante los tres segundos.

Si se realiza una espera bloqueante, la tarea cede el hilo de ejecución y al solo haber dos tareas la segunda termina siendo ejecutada sin importar las prioridades de estas.

Con la espera no bloqueante, la tarea solo cede el hilo de ejecución de manera preventiva cada *tick* del sistema operativo. Cuando las prioridades son iguales sabemos que las dos tareas mantienen control equitativo del hilo de ejecución. Pero cuando la prioridad de la misma se incrementa (gracias al *polling* del botón), el *scheduler* le devuelve el control, produciendo el caso de *starvation* necesario para congelar el segundo led.

El manejo del tiempo se pudo realizar con llamadas a **HAL_GetTick** y una variable de control para sólo mantener cuenta del tiempo cuando sea necesario. El *jitter* introducido por mantener registro del tiempo sólo cuando se requiere es despreciable, y necesario para el funcionamiento correcto de la tarea.

Desafío 6: Sincronización Unidireccional (Deferred Interrupt Processing)

Código relevante

```
SemaphoreHandle_t semChallenge6 = NULL;

void vTask1Challenge6(void *pvParameters){
    uint16_t status;
    while(1){
        status = xSemaphoreTake(semChallenge6, 0);
        if(status == pdTRUE) {
            HAL_GPIO_TogglePin(GPIOA, LD3_Pin);
        }
    }
}

void vChallenge6(){
    const toggleInstruction_t taskInstruction = {GPIOA, LD4_Pin, 0};
    xTaskCreate(vTask1Challenge6, "Task1", 100, (void *)
    &taskInstruction, 1, NULL);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    xSemaphoreGiveFromISR(semChallenge6, NULL);
}
```

Análisis

Proponer una forma de visualizar dinamicamente (algo cambia todo el tiempo) que la tarea en cuestion efectivamente no esta consumiendo tiempo del procesador. No esta permitido crear nuevas tareas.

La visualización se puede hacer en la consola (imprimiendo algo), o en la misma placa (conmutando otro led distinto al usado).

La lógica sería la misma para ambos tipos: para la sincronización por *polling*, la visualización ocurriría antes de hacer la lectura del pin correspondiente al botón, y para la sincronización por interrupción, luego de tomar el semáforo.

Esto causaría dos efectos distintos: para la tarea de *polling*, la impresión (o conmutación del led) ocurriría periódicamente de acuerdo al tiempo de respuesta que se estableció. En el ejemplo del desafío 3, sucedería cada 10ms.

Para la tarea con semáforos, la visualización ocurriría junto a la pulsación del botón. En el caso de la conmutación del led, ambos estarían sincronizados.

Desafío 7: Secuenciamiento de Tareas (Task Synchronization)

Código relevante

```
SemaphoreHandle_t sem1Ch7 = NULL;
SemaphoreHandle_t sem2Ch7 = NULL;
SemaphoreHandle_t sem3Ch7 = NULL;

void vTask1Ch7(void *pvParameters){
    uint16_t status;
    while(1){
        status = xSemaphoreTake(sem3Ch7, (TickType_t) 10);
        if(status == pdTRUE){
            HAL_GPIO_WritePin(GPIOD, LD5_Pin, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOD, LD3_Pin, GPIO_PIN_SET);
            vTaskDelay(pdMS_TO_TICKS(500));
            status = xSemaphoreGive(sem1Ch7);
        }
    }
}

void vTask2Ch7(void *pvParameters){
    uint16_t status;
    while(1){
        status = xSemaphoreTake(sem1Ch7, (TickType_t) 10);
        if(status == pdTRUE){
            HAL_GPIO_WritePin(GPIOD, LD3_Pin, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOD, LD4_Pin, GPIO_PIN_SET);
            vTaskDelay(pdMS_TO_TICKS(500));
            status = xSemaphoreGive(sem2Ch7);
        }
    }
}

void vTask3Ch7(void *pvParameters){
    uint16_t status;
    while(1){
        status = xSemaphoreTake(sem2Ch7, (TickType_t) 10);
        if(status == pdTRUE){
            HAL_GPIO_WritePin(GPIOD, LD4_Pin, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOD, LD5_Pin, GPIO_PIN_SET);
            vTaskDelay(pdMS_TO_TICKS(500));
            status = xSemaphoreGive(sem3Ch7);
        }
    }
}

void vChallenge7(){
    sem1Ch7 = xSemaphoreCreateBinary();
    sem2Ch7 = xSemaphoreCreateBinary();
    sem3Ch7 = xSemaphoreCreateBinary();

    xSemaphoreGive(sem3Ch7);

    xTaskCreate(vTask1Ch7, "Task", 100, NULL, 1, NULL);
    xTaskCreate(vTask2Ch7, "Task", 100, NULL, 1, NULL);
    xTaskCreate(vTask3Ch7, "Task", 100, NULL, 1, NULL);
}
```